

Where Does Meta-Learning Help a Quadrotor? Seven Algorithms Combined with Classical Control and Planning

Amal Mehta

TODO: affiliation

amal.mehta@gmail.com

Abstract—A quadrotor deployed outside the lab meets conditions it was not tuned for: extra payload, a worn motor, wind and command latency. Meta-reinforcement learning promises adaptation from a few flights, while classical stacks built on geometric control, adaptive control, occupancy mapping and A* planning are dependable but fixed. We ask where meta-learning actually helps when it is combined with such a stack. In one simulator parameterised after a Holybro X500 V2, we insert learning at three points: a residual on the geometric controller, the controller’s gains, and the local planner of a depth-camera, mapping and A* navigation stack. We compare seven algorithms (MAML, FOMAML, ANIL, Meta-SGD, Reptile, PEARL and RL²) with nominal, L1-adaptive, domain-randomised and fine-tuning baselines on held-out and out-of-distribution dynamics. A learned residual cuts held-out tracking error from 15.5 cm with L1 adaptation to 7.3 cm, but gradient-based adaptation adds little beyond domain randomisation. Meta-learning helps most when it tunes classical gains (Meta-SGD: 15.6 to 11.3 cm). In navigation, gradient-based adaptation degrades learned planners while RL² improves with experience, and the classical planner remains the most reliable on the full stack in unseen rooms. Outside the training distribution every method fails, and learned residuals crash more often than the plain controller. We release the suite and recommend a hardware platform and staged plan for real-world tests.

I. INTRODUCTION

Small aerial robots rarely fly the vehicle they were tuned on. A delivery adds payload, a propeller chips, the wind picks up and the companion computer adds latency. Classical control handles part of this: a geometric tracking controller [1] with integral action rejects slow disturbances, and L1 adaptive augmentation estimates and cancels a lumped disturbance online [2]. Learning-based adaptation goes further: meta-learning trains a model that adapts from a few samples of a new task [3], and learned adaptation has produced agile flight in strong winds [4] and a single controller for very different quadrotors [5].

Two gaps motivate this work. First, meta-reinforcement-learning algorithms are usually compared with each other on locomotion benchmarks, not against classical controllers and planners that run in the same loop. Second, a deployed autonomy stack has several places where learning could enter (the low-level controller, its gains, the local planner), and it is not clear which one benefits.

We study both in one simulation suite. A batched quadrotor model parameterised after a Holybro X500 V2 runs a classical

stack: a geometric controller with collective-thrust and body-rate (CTBR) output, a simulated RealSense D435i depth camera, a log-odds occupancy map and an A* planner (Fig. 1). Meta-learning plugs into three places: a residual on the controller (Combination 1), the controller’s gains (Combination 2) and the local planner (Combination 3). Each must adapt to a new task drawn from payload, motor wear, drag, wind, gusts and latency (Table I).

Our contributions are:

- an open simulation suite with the three classical+meta combinations (Sec. III);
- a benchmark of seven meta-learning algorithms against nominal, L1-adaptive, domain-randomised (DR) and fine-tuning baselines on held-out and out-of-distribution (OOD) tasks (Sec. VI);
- findings on where adaptation helps and where it hurts: it helps most when tuning classical gains, adds little to a learned residual, degrades learned navigation planners except RL², and none of the methods survives the OOD tasks;
- a hardware recommendation and staged sim-to-real plan for testing the stack (Sec. VII).

Code, results and a replay of recorded flights are at <https://github.com/amalmehta/DroneAutonomy>.

II. RELATED WORK

Meta-reinforcement learning. Gradient-based methods learn an initialisation from which a few policy-gradient steps adapt to a new task: MAML [3], its first-order approximation FOMAML, Reptile [6], ANIL, which adapts only the output layer [7], and Meta-SGD, which also learns per-parameter step sizes [8]. Context-based methods instead infer the task from experience: PEARL encodes recent transitions into a latent variable for an off-policy actor-critic [9], and RL² uses a recurrent policy whose state carries what it has learned across episodes [10], [11]. These are usually compared with each other on simulated locomotion; we compare them inside a classical flight stack and against classical adaptive control.

Learned adaptation for flight. Neural-Fly learns wind-invariant features offline and adapts a linear layer online [4]; adaptive-control-oriented meta-learning trains features for an adaptive law [12]; rapid motor adaptation infers latent dynamics from history [13], an idea carried to quadrotors of very

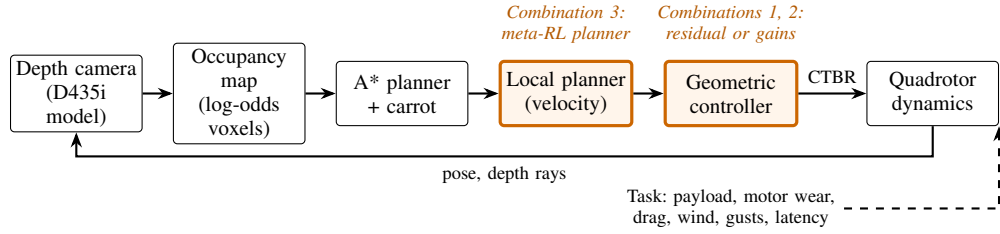


Fig. 1. Meta-learning plugs into three places of one classical stack. The local planner (Combination 3) turns depth and a carrot point on the A* path into a velocity command. The geometric controller turns that into collective thrust and body rates, with a learned residual (Combination 1) or meta-learned gains (Combination 2). The task perturbs the dynamics.

different sizes [5]. Domain randomisation trains a single robust policy instead [14], [15], [16]. L1 adaptive augmentation of a geometric controller is a strong classical alternative [2]. We include domain-randomised and L1 baselines next to the meta-learners.

Residual and hybrid control. Residual RL learns a correction on top of a fixed controller [17], [18]. Our Combinations 1 and 2 are residual and gain-tuning variants of this idea, trained with meta-learning.

Learned versus classical navigation. Classical stacks build a map, for example an OctoMap [19], and plan on it with search such as A* [20]. Learned policies can fly fast through clutter from depth alone [21], and RL can outperform optimal control in racing [22]. We keep the map and planner and ask whether a meta-learned local planner improves on a classical one. Open simulators and platforms for learning-based flight include gym-pybullet-drones [23] and Agilicious [24].

III. SYSTEM

A. Vehicle model

We simulate many drones at once, each with its own task, in a batched rigid-body model integrated at 500 Hz. Commands are collective thrust and body rates (CTBR), the setpoint PX4 accepts in offboard mode [25]. Each command passes through a latency buffer, then a PX4-like body-rate PI loop that uses the *nominal* inertia, then an X-configuration mixer that gives up yaw torque first when motors saturate. First-order motors (time constant 40 ms) are scaled by per-motor efficiencies. Forces include linear drag (0.3 Ns/m), steady wind and Ornstein–Uhlenbeck gusts. The nominal vehicle approximates a Holybro X500 V2 carrying a Jetson and a D435i: mass 2.0 kg, arm 0.25 m and inertia 0.0217/0.0217/0.040 kg m² from PX4’s x500 simulation model; the thrust-to-weight ratio of 2.1, motor lag and drag are our estimates (Sec. VII).

B. Tasks

A task fixes payload (mass scale; inertia grows by half the relative mass increase, as for a payload near the centre), the efficiency of one randomly chosen motor, drag, steady wind, gust intensity and command latency (Table I). Training and held-out tasks share ranges but use disjoint random seeds. OOD tasks lie beyond every range at once.

TABLE I
TASK DISTRIBUTION. EACH TASK DRAWS EVERY PARAMETER FROM THE GIVEN RANGE.

Parameter	Unit	Train / held-out	OOD
Payload (mass scale)	×	0.85–1.30	1.30–1.40
Weakest motor efficiency	%	75–100	65–75
Drag scale	×	0.5–2.0	2.0–3.0
Steady wind	m/s	0.0–2.5	2.5–4.0
Gust std. (OU, 0.5 s)	m/s	0.0–0.8	0.8–1.5
Command latency	ms	0–30	30–50

C. Classical control

A geometric controller [1] runs at 100 Hz: a PID position loop produces a desired acceleration, limited to 35° tilt, which sets collective thrust and an attitude whose error gives body-rate commands. It has seven gains: proportional, derivative and integral gains for the horizontal and vertical axes, and an attitude gain. The adaptive baseline adds piecewise-constant L1 augmentation [2]: a velocity predictor estimates a lumped acceleration disturbance, whose low-pass-filtered value is subtracted from the desired acceleration.

Tracking. Combinations 1 and 2 are scored on trajectory tracking: each episode follows a smooth random reference (a sum of two cosines per axis, at most 2 m/s horizontally; 15% of episodes are pure hover) for 4 s. The reward is $\exp(-\|e\|/0.25)$ for position error e in metres, minus a small action penalty. An episode ends early if the error exceeds 1.5 m or the tilt exceeds 70°.

Combination 1: residual on the controller. At 50 Hz a policy adds a bounded correction to the controller’s output: up to 25% of hover thrust, and up to 1 rad/s in roll and pitch rate (0.5 rad/s in yaw). It observes tracking errors, velocity, attitude, body rates, the reference acceleration, its previous action and the controller’s own command.

Combination 2: controller gains. The policy is a Gaussian over the logarithm of the seven gain scales relative to nominal. One gain vector is drawn per episode and the controller flies the whole episode with it.

D. Navigation stack (Combination 3)

Rooms measure 14×8×3 m and contain randomly placed pillars and boxes; a quarter of the boxes hang from the ceiling with their lowest edge at 1.9 m or higher, so the map and

planner work in three dimensions. Start and goal sit at opposite ends. The drone is a 0.35 m sphere for collision checks.

Perception and mapping. A depth camera modelled on the D435i casts rays over an $87^\circ \times 58^\circ$ field of view up to 5 m, adds noise with standard deviation $0.008 d^2$ and drops 2% of pixels. The policy sees a 16×8 image; the mapper uses 48×24 . A log-odds voxel map with 0.2 m voxels carves free space along each ray, stopping one voxel short of a hit so that grazing rays don’t erase surfaces.

Planning. A* searches the 26-connected voxel grid inflated by the drone radius plus a 0.15 m margin, replanned at 2 Hz. The carrot is the furthest point up to 1.5 m along the path that is still in straight-line sight.

Local planner. At 10 Hz the local planner outputs a velocity command in the heading frame. The classical controller tracks it through a leashed position setpoint while yaw turns toward the carrot. The learned planner observes the depth image, the carrot and goal in the heading frame, velocity, attitude, altitude and its previous action. It is rewarded for progress along the true shortest path, with penalties for time, proximity, jerky commands and collisions, and a bonus at the goal. The classical local planner pursues the carrot, slows down near depth returns and adds a repulsive term from close returns. For speed, learned planners are trained with a privileged planner: a distance field computed on the true map, equivalent to A* on a perfect map. Evaluation also runs the full online stack.

IV. LEARNERS AND BASELINES

All step policies are Gaussian MLPs (64–64 units for tracking, 128–64 for navigation) with a state-independent standard deviation. To keep differences down to the meta-learning rule, the gradient-based methods share one estimator: per-task linear feature baselines with generalised advantage estimation [26] and PPO-clipped updates [27].

MAML family. Each iteration samples 16 tasks and flies 10 episodes per task. One vanilla policy-gradient step with step size 0.1 adapts the policy to each task; all tasks adapt in one batched backward pass. The adapted policies fly 10 new episodes, and a PPO-clipped outer loss on those episodes is backpropagated through the adaptation step. MAML [3] differentiates through the inner gradient (second order); FOMAML drops that term; ANIL adapts only the output layer and the standard deviation [7]; Meta-SGD also learns a step size for every parameter [8]. Like the original MAML for RL, we ignore how the pre-adaptation sampling distribution depends on the parameters [28].

Reptile [6] runs three rounds of per-task PPO (Adam, four epochs per round), each on fresh episodes, then moves the shared parameters toward the mean adapted parameters with a step that anneals from 0.5 to 0.1.

PEARL [9] trains a soft actor-critic [29] conditioned on a five-dimensional latent task variable. A permutation-invariant encoder maps transitions from 32 training tasks to a product-of-Gaussians posterior, trained through the critic loss plus a KL penalty to the prior.

RL² [10], [11] uses a GRU policy with 128 units that sees the previous action, reward and an episode-start flag, and keeps its hidden state over a trial of three episodes on the same task. It is trained with recurrent PPO over whole trials.

Baselines. *Classical* uses nominal gains and no learning; *Classical + L1* adds L1 augmentation. *DR* trains one policy with PPO over the whole task distribution (domain randomisation) and never adapts; *DR + fine-tune* adapts it at test time with the same per-task PPO rounds as Reptile. For Combination 2 the DR baseline is the same Gaussian over gains, trained over all tasks without a meta objective. For navigation we add *end-to-end DR*, a DR planner that sees the goal direction instead of the carrot, with no map or planner.

Navigation specifics. Training navigation planners from scratch is the most expensive part of the study, so to fit our CPU budget the gradient-based meta-learners start from the trained DR planner and meta-train from there. PEARL and RL² use different networks and train from scratch, so their comparison with the others is confounded by starting point. With the tracking step size (Adam, 3e-3), per-task PPO adaptation wrecked the warm-started navigation planner (Reptile reached 0% success), so navigation uses 3e-4 for both Reptile and DR + fine-tune; the diverged run is kept in the released results.

V. EXPERIMENTAL PROTOCOL

Every method is scored on fixed task sets: 32 held-out and 32 OOD tasks for tracking, and 24 of each for navigation, flown in rooms from a test pool never used in training (128 rooms per pool). Before adapting and after each of three adaptation stages, we fly deterministic evaluation episodes (the policy mean) on every task. A stage of gradient-based adaptation uses 10 exploratory episodes and one update, so three stages use 30 episodes. A PEARL stage adds one episode of context, and an RL² stage is one more episode of the trial. Non-adapting methods are evaluated once. We report tracking RMSE in centimetres (an early termination counts as 1.5 m error for the rest of the episode) and navigation success within 0.5 m of the goal before a collision or a 30 s time-out.

Navigation is scored twice. Learned planners are scored with the privileged planner, and then on the full online stack (depth camera, map built in flight, A*) in 12 unseen rooms per split, before and after one adaptation stage. That stage’s exploration flights use the privileged planner, like a calibration flight in a known room.

Tracking methods are trained with 2 seeds and navigation methods with 1, and the deterministic classical baselines are evaluated once. Intervals are 95% normal intervals over tasks and seeds. Everything ran on one shared Intel Core i9-9980HK CPU without a GPU.

VI. RESULTS

A learned residual beats classical adaptation. On held-out tasks the nominal geometric controller tracks with 21.4 cm RMSE and L1 augmentation with 15.5 cm (Table II). A DR

TABLE II

TRACKING UNDER NEW DYNAMICS. A LEARNED RESIDUAL BEATS CLASSICAL AND L1 CONTROL BY A WIDE MARGIN, WHILE META-LEARNING HELPS MOST WHEN IT TUNES THE CONTROLLER’S GAINS. HELD-OUT RMSE BEFORE AND AFTER ADAPTING (MEAN, WITH THE 95% HALF-INTERVAL AFTER ADAPTING), OOD RMSE AND OOD CRASH RATE AFTER ADAPTING. DATA: ADAPTATION EPISODES USED. BOLD: BEST IN COLUMN. TRACKING METHODS USE 2 TRAINING SEEDS.

Method	Data (ep.)	Combination 1: residual on controller				Combination 2: controller gains		
		Before (cm)	After (cm)	OOD (cm)	OOD crash (%)	Before (cm)	After (cm)	OOD (cm)
<i>Classical</i>								
Classical (nominal gains)	0	21.4	21.4	78.7	34	21.4	21.4	78.7
Classical + L1	0	15.5	15.5	72.1	52	–	–	–
<i>Non-meta learned</i>								
DR (no adapt.)	0	7.3	7.3	63.1	44	14.4	14.4	76.9
DR + fine-tune	30	7.3	8.0±1.0	63.6	43	14.4	14.2±1.9	75.5
<i>Gradient meta-learning</i>								
MAML	30	8.7	7.8±1.1	70.8	49	15.6	13.7±2.7	69.8
FOMAML	30	8.5	7.8±1.0	66.9	46	15.7	13.9±2.8	72.6
ANIL	30	8.6	7.7±1.1	70.4	49	–	–	–
Meta-SGD	30	8.7	7.8±1.1	69.3	48	15.6	11.3±2.0	69.9
Reptile	30	15.2	14.8±2.0	68.6	36	13.5	13.4±1.7	75.4
<i>Context meta-learning</i>								
PEARL	3	10.5	9.0±1.1	69.2	49	–	–	–
RL ²	3	18.4	17.4±1.5	64.6	36	–	–	–

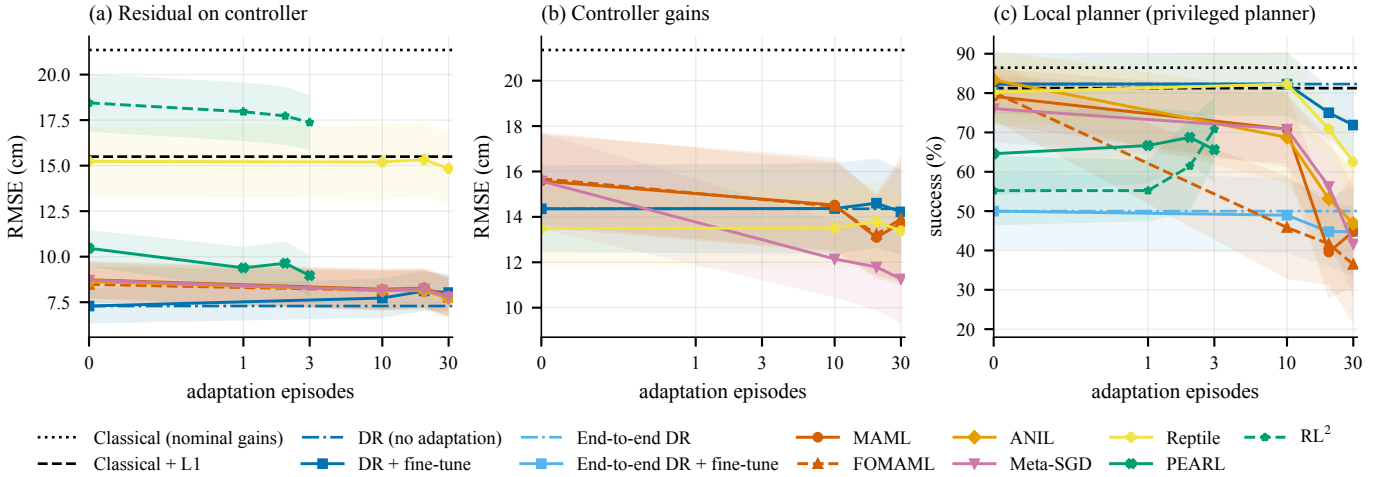


Fig. 2. Adaptation helps when it tunes gains (b), barely moves a learned residual (a) and degrades learned navigation planners (c), where only RL² improves with experience. Held-out tasks; mean and 95% interval over tasks and seeds; horizontal lines are methods that do not adapt. PEARL and RL² use one episode per stage, the gradient-based methods ten.

residual trained over all tasks reaches 7.3 cm without any adaptation, 2.9 $\times$ and 2.1 $\times$ lower respectively.

Gradient-based meta-learning adds little to the residual.

MAML, FOMAML, ANIL and Meta-SGD start slightly worse than DR (8.7, 8.5, 8.6 and 8.7 cm) and improve with 30 adaptation episodes to 7.8, 7.8, 7.7 and 7.8 cm, within the interval of DR’s 7.3 cm (Fig. 2a). Fine-tuning DR with PPO makes it slightly worse (8.0 cm), and Reptile trails at 14.8 cm. With a broad task distribution and a strong classical controller underneath, a single robust residual already captures most of the benefit in our setup.

Meta-learning helps most when it tunes gains. When the policy chooses the controller’s seven gains (Combination 2), adaptation matters more: Meta-SGD improves from 15.6 to

11.3 cm, and MAML and FOMAML reach 13.7 and 13.9 cm. DR-tuned gains stay at 14.4 cm, or 14.2 cm with fine-tuning (Fig. 2b). This result is preliminary: with two seeds, Meta-SGD’s interval (± 2.0 cm) only just separates from the fine-tuned DR gains (± 1.9 cm). Tuned gains remain well above the learned residual (7.3 cm), but a gain vector is a far smaller and more interpretable thing to adapt on a real vehicle.

Context-based methods adapt from fewer episodes.

PEARL improves from 10.5 to 9.0 cm with three episodes of context, against the 30 episodes the gradient methods use, though it does not reach their final error. RL² improves from 18.4 to 17.4 cm and is the weakest learned residual.

Adaptation hurts learned navigation planners, except RL². With the privileged planner, every warm-started gradient

TABLE III

NAVIGATION IN UNSEEN ROOMS. THE CLASSICAL LOCAL PLANNER IS THE MOST RELIABLE ON THE FULL STACK; GRADIENT-BASED ADAPTATION LOWERS LEARNED PLANNERS’ SUCCESS, WHILE RL^2 IMPROVES. SUCCESS AND COLLISION RATES BEFORE AND AFTER ADAPTING, WITH THE PRIVILEGED PLANNER (24 ROOMS, 3 STAGES) AND ON THE FULL ONLINE-MAPPING STACK (12 ROOMS, 1 STAGE), AND OOD SUCCESS. ONE TRAINING SEED. BOLD: BEST IN COLUMN.

Method	Data (ep.)	Privileged planner, 24 rooms			Full stack, 12 rooms			OOD
		Before (%)	After (%)	Coll. (%)	Before (%)	After (%)	Coll. (%)	After (%)
<i>Classical</i>								
Classical (nominal gains)	0	86	86	12	83	83	0	14
Classical + L1	0	81	81	18	75	75	0	11
<i>Non-meta learned</i>								
DR (no adaptation)	0	82	82	18	67	67	33	15
DR + fine-tune	30	82	72	28	67	67	33	21
End-to-end DR	0	50	50	50	42	42	58	7
End-to-end DR + fine-tune	30	50	45	55	42	42	58	9
<i>Gradient meta-learning (warm-started)</i>								
MAML	30	79	45	52	58	58	42	7
FOMAML	30	80	36	61	58	75	25	8
ANIL	30	83	47	50	67	50	50	8
Meta-SGD	30	76	42	52	75	33	58	6
Reptile	30	80	62	38	58	50	50	17
<i>Context meta-learning (from scratch)</i>								
PEARL	3	65	66	33	67	17	83	7
RL ²	3	55	71	28	33	50	50	12

method loses success as it adapts: MAML from 79% to 45%, FOMAML from 80% to 36%, ANIL from 83% to 47%, Meta-SGD from 76% to 42% and Reptile from 80% to 62% (Table III, Fig. 2c). DR with fine-tuning also drops, from 82% to 72%. Collision rates rise as success falls, so the updates trade safety away. RL^2 is the only method that improves with experience, from 55% to 71%, and PEARL stays flat (65% to 66%). We read this as noisy policy gradients from 10 sparse-reward episodes per task overwriting a competent planner, while RL^2 adapts without gradient steps. This is one seed, so the effect needs confirming.

On the full stack, the classical planner is most reliable. With a map built in flight and A* replanning, the classical local planner reaches the goal in 83% of the 12 unseen rooms with 0% collisions. The learned planners reach between 17% and 75% with 25–83% collisions. With 12 rooms one room moves a rate by 8 points, so we treat these numbers as indicative: the classical planner was never worse, and it never collided. Fig. 3 shows a recorded classical flight.

The map and planner matter. The end-to-end DR planner, which sees only the goal direction, succeeds in 50% of the 24 held-out rooms, against 82% for the same learner given the carrot, and in 42% against 67% on the full stack.

Outside the training distribution every method fails. On the OOD tasks, which combine heavier payload, weaker motors, stronger wind and longer latency, no tracking method gets below 63.1 cm RMSE and no navigation method exceeds 21% success. Learned residuals also crash more often (43–49%) than the nominal controller (34%). Adaptation does not rescue them: three stages change OOD error by at most 6 cm. Our OOD set is harsh, so it measures how methods fail more than how they adapt, but the ordering is a caution for deployment: on the edge of the envelope, the plain classical

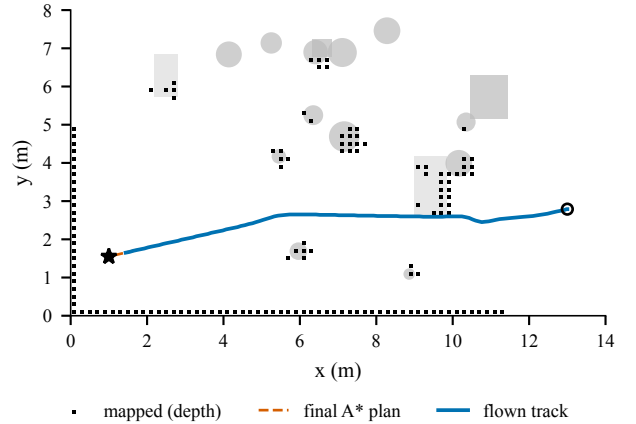


Fig. 3. The classical stack maps an unseen room from its depth camera and reaches the goal. Grey: true obstacles (paler boxes hang overhead and are flown under); black: voxels mapped by the end of the flight; dashed: final A* plan; blue: flown track from start (circle) to goal (star).

controller degraded most gracefully.

VII. REAL-WORLD PLATFORM AND TEST PLAN

The stack needs a flight controller that accepts CTBR or velocity setpoints from a companion computer at 50Hz or more, a depth camera, onboard compute for mapping and planning, and a vehicle that is safe in a netted cage. Table IV compares indoor research platforms in the \$1–3k range, from manufacturer pages checked on 2026-10-05.

We recommend a Holybro X500 V2 with a Jetson Orin Nano Super and a RealSense D435i, about \$1.6–2.0k in parts. PX4’s offboard mode takes body rates and collective thrust directly, the policies’ own action space. The D435i is the

TABLE IV
CANDIDATE PLATFORMS. ONLY THE X500 BUILD IS IN STOCK, WITHIN BUDGET, AND CARRIES A DEPTH CAMERA THAT MATCHES THE SIMULATOR. STATUS FROM MANUFACTURER STORES ON 2026-10-05.

Platform	Mass	Depth	CTBR	Status
X500 V2 build [†]	≈2.0 kg	D435i	PX4	in stock
Crazyflie 2.1 Brushless	37 g	none	partial	listed
Starling 2	285 g	ToF	PX4	unavailable new
Starling 2 Max	566 g	ToF (C29 only)	PX4	over budget
PX4 Vision V1.5	893 g*	discontinued	PX4	sold out

[†]Holybro X500 V2 + Jetson Orin Nano Super + D435i. *Without battery.

camera the simulator models, and the X500 is PX4’s reference airframe, so its simulator parameters come from a published model. The cost of this choice is a 2 kg vehicle with 10-inch propellers, which must fly inside a netted cage. The Crazyflie is the safest vehicle for checking that low-level control transfers, but it cannot carry a depth camera.

The test plan has five stages, each gated by the previous one: (0) system identification of mass, inertia, thrust curve and motor lag to replace our estimates; (1) hover with the policy in the loop under motion capture; (2) trajectory tracking with known added payloads, compared with the simulator; (3) known foam obstacles with live depth mapping; (4) an unknown room with onboard visual-inertial odometry only. Safety layers include an RC kill switch, PX4 geofence, offboard-loss failsafe, and a watchdog that falls back to hold if commands arrive late.

VIII. LIMITATIONS

All results are in simulation; no real flights have been made. The thrust-to-weight ratio, motor lag and drag of the simulated vehicle are estimates. The aerodynamics are simple (linear drag, no ground effect or rotor drag), the depth model lacks texture- and lighting-dependent stereo artefacts, and state estimation is perfect. Compute was a single shared CPU, so tracking uses 2 seeds and navigation 1, and the full-stack navigation score rests on 12 rooms per split. The navigation comparison between warm-started gradient methods and from-scratch PEARL and RL² is confounded by starting point. The MAML implementations ignore the dependence of the pre-adaptation sampling distribution on the parameters. Finally, our OOD tasks proved harsh enough that every method fails on them.

IX. CONCLUSION

Combining meta-learning with a classical quadrotor stack at three insertion points, we found that a robust learned residual captures most of the tracking benefit without meta-learning, that meta-learning pays off most when it tunes classical gains, and that gradient-based adaptation can make learned navigation planners less safe, while a recurrent policy improved with experience. A classical local planner on an online map was the most reliable navigator in unseen rooms, and every method failed beyond the training distribution. These results argue for keeping classical components in the loop and adapting

their parameters, and for testing adaptation methods on out-of-distribution conditions before trusting them on a real vehicle, which is the next step on the platform we recommend.

ACKNOWLEDGMENT

The simulator, learning algorithms, experiments, benchmark, figures and table scripts, and the text of this paper were produced with an AI coding assistant (Claude Code, Anthropic) working under the author’s direction. Citation metadata were checked against arXiv and Crossref by script. The author reviewed the results and is responsible for all claims.

REFERENCES

- [1] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor UAV on SE(3),” in *IEEE Conference on Decision and Control (CDC)*, 2010.
- [2] Z. Wu, S. Cheng, K. A. Ackerman, A. Gahlawat, A. Lakshmanan, P. Zhao, and N. Hovakimyan, “L1 adaptive augmentation for geometric tracking control of quadrotors,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.
- [3] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *International Conference on Machine Learning (ICML)*, 2017.
- [4] M. O’Connell, G. Shi, X. Shi, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, “Neural-fly enables rapid learning for agile flight in strong winds,” *Science Robotics*, 2022.
- [5] D. Zhang, A. Loquercio, X. Wu, A. Kumar, J. Malik, and M. W. Mueller, “Learning a single near-hover position controller for vastly different quadcopters,” *arXiv preprint arXiv:2209.09232*, 2022.
- [6] A. Nichol, J. Achiam, and J. Schulman, “On first-order meta-learning algorithms,” *arXiv preprint arXiv:1803.02999*, 2018.
- [7] A. Raghu, M. Raghu, S. Bengio, and O. Vinyals, “Rapid learning or feature reuse? towards understanding the effectiveness of maml,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [8] Z. Li, F. Zhou, F. Chen, and H. Li, “Meta-sgd: Learning to learn quickly for few-shot learning,” *arXiv preprint arXiv:1707.09835*, 2017.
- [9] K. Rakelly, A. Zhou, D. Quillen, C. Finn, and S. Levine, “Efficient off-policy meta-reinforcement learning via probabilistic context variables,” in *International Conference on Machine Learning (ICML)*, 2019.
- [10] Y. Duan, J. Schulman, X. Chen, P. L. Bartlett, I. Sutskever, and P. Abbeel, “RL²: Fast reinforcement learning via slow reinforcement learning,” *arXiv preprint arXiv:1611.02779*, 2016.
- [11] J. X. Wang, Z. Kurth-Nelson, D. Tirumala, H. Soyer, J. Z. Leibo, R. Munos, C. Blundell, D. Kumaran, and M. Botvinick, “Learning to reinforcement learn,” *arXiv preprint arXiv:1611.05763*, 2016.
- [12] S. M. Richards, N. Azizan, J.-J. Slotine, and M. Pavone, “Adaptive-control-oriented meta-learning for nonlinear systems,” in *Robotics: Science and Systems (RSS)*, 2021.
- [13] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Robotics: Science and Systems (RSS)*, 2021.
- [14] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [15] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.
- [16] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, “Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors,” *arXiv preprint arXiv:1903.04628*, 2019.
- [17] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [18] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.

- [19] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [20] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [21] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, 2021.
- [22] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, 2023.
- [23] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, "Learning to fly—a gym environment with PyBullet physics for reinforcement learning of multi-agent quadcopter control," *arXiv preprint arXiv:2103.02142*, 2021.
- [24] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, 2022.
- [25] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.
- [26] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.
- [27] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [28] J. Rothfuss, D. Lee, I. Clavera, T. Asfour, and P. Abbeel, "ProMP: Proximal meta-policy search," *arXiv preprint arXiv:1810.06784*, 2018.
- [29] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018.